

SOFTWARE DESIGN PATTERN

Introduction & Foundations

Problem → Principles → Design → Refactoring → Pattern

Md Saeed Siddik

Assistant Professor, IIT, University of Dhaka

<https://sites.google.com/view/saeedsiddik>

What Is Software Design?

The process of deciding what components should exist, what responsibilities each has, how objects communicate, and how the system will absorb future change.

Ask these questions early:

- What happens when PayPal / bKash is added?
- What if SMS notifications are required?
- What if the database engine changes?

```
// naive design: everything in one class
class Order {
    calculateTotal()
    processPayment()
    sendEmail()
    generateInvoice()
    saveToDatabase()
    printReceipt()
}
```

It works at first: but every new requirement forces a change inside this one class.

FOUNDATIONS

Why Does Software Design Matter?

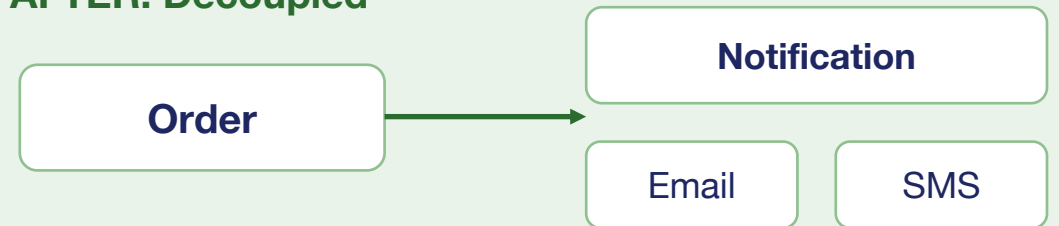
Requirements evolve: email today, SMS tomorrow, WhatsApp next. Design decides how painful that evolution is.

BEFORE: Tightly Coupled



Adding SMS means editing Order directly.

AFTER: Decoupled



Order no longer knows the exact channel.

Good design makes change easier.

Developers Keep Hitting the Same Problems

1

Object Creation

How do we create objects without coupling clients to concrete classes?

2

Incompatible Interfaces

How do we make two mismatched classes work together?

3

Adding Responsibilities

How do we extend behavior without editing the original class?

4

Complex Subsystems

How do we offer a simple interface to a complicated system?

5

Object Communication

How do multiple objects get notified when something changes?

These recurring problems: and their recurring solutions: are the basis of design patterns.

DEFINITION

What Is a Design Pattern?

A reusable, general solution to a commonly occurring software design problem.

A pattern is NOT copy-paste code: it's a design template.

Participants

Relationships

Communication

Advantages

Limitations

**Applicable
Situations**

A pattern describes each of these: for example, the Observer pattern for “notify multiple objects on change.”

Pattern, Algorithm, Framework, or Library?

Concept	What It Solves / Provides	Example
Algorithm	Step-by-step procedure for a computational task	<i>Binary Search</i>
Design Pattern	Structure & relationships for a recurring design problem	<i>Factory Method</i>
Framework	Reusable structure you build an app inside: may use patterns internally	<i>Spring, Django, .NET</i>
Library	Reusable code you call: provides code, not a design idea	<i>Java Collections, NumPy</i>

A Brief History of Design Patterns

1

Architecture

Christopher Alexander describes recurring solutions to building-design problems.

2

1987

Kent Beck & Ward Cunningham apply the idea of patterns to object-oriented programming.

3

1994

Gamma, Helm, Johnson & Vlissides: the “Gang of Four”: publish Design Patterns: Elements of Reusable Object-Oriented Software, with 23 classic patterns.

The 23 GoF Patterns

Creational

5

- Abstract Factory
- Builder
- Factory Method
- Prototype
- Singleton

Structural

7

- Adapter
- Bridge
- Composite
- Decorator
- Facade
- Flyweight
- Proxy

Behavioral

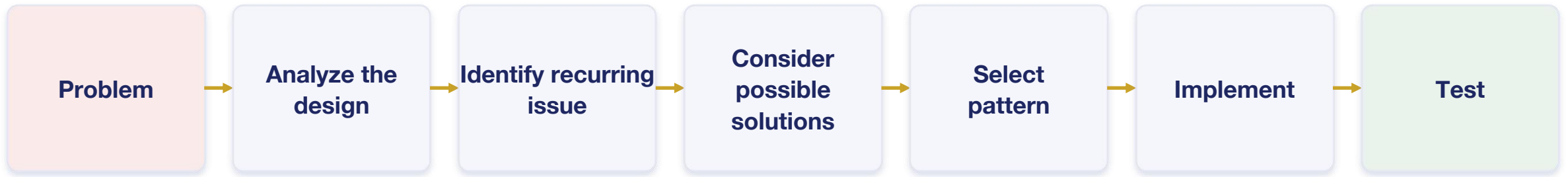
11

- Chain of Responsibility
- Command
- Interpreter
- Iterator
- Mediator
- Memento
- Observer
- State
- Strategy
- Template Method
- Visitor

This course covers Creational and Structural patterns before the midterm.

When Should We Use a Design Pattern?

Never use a pattern just because “patterns are good.” Follow the problem.



A pattern should solve a real problem: not create unnecessary complexity.

SOLID Principles

S

Single Responsibility

A class should have one responsibility, one reason to change.

O

Open/Closed

Open for extension, closed for modification: avoid growing if/else chains.

L

Liskov Substitution

A subtype must be usable wherever its base type is expected.

I

Interface Segregation

Don't force clients to depend on methods they don't use.

D

Dependency Inversion

Depend on abstractions, not concrete implementations.

DRY, KISS & YAGNI

DRY

Don't Repeat Yourself

Avoid duplicating logic: but don't over-abstract to remove every similarity.

KISS

Keep It Simple

Avoid unneeded complexity. Simplicity should be appropriate, not minimal at all costs.

YAGNI

You Aren't Gonna Need It

Don't build for hypothetical future requirements: build for the real one.

How Principles Lead to Patterns



Worked Example



Patterns are tools, not goals.

Code Smells

1

Long Method

A function doing far too much

3

Duplicate Code

Same logic copied in multiple places

5

Switch / If-Else Explosion

Growing conditional chains

7

Feature Envy

A class constantly reaches into another's data

2

Large / God Class

One class owns every responsibility

4

Long Parameter List

Methods needing many arguments

6

Tight Coupling

One class knows too much about another

Refactoring: Structure Changes, Behavior Doesn't

Refactoring changes a program's internal structure without changing what it does externally.

```
void generateInvoice() {  
    // calculate tax  
    // calculate discount  
    // format invoice  
    // save invoice  
    // send email  
}
```

```
void generateInvoice() {  
    calculateTax();  
    calculateDiscount();  
    formatInvoice();  
    saveInvoice();  
    sendEmail();  
}
```

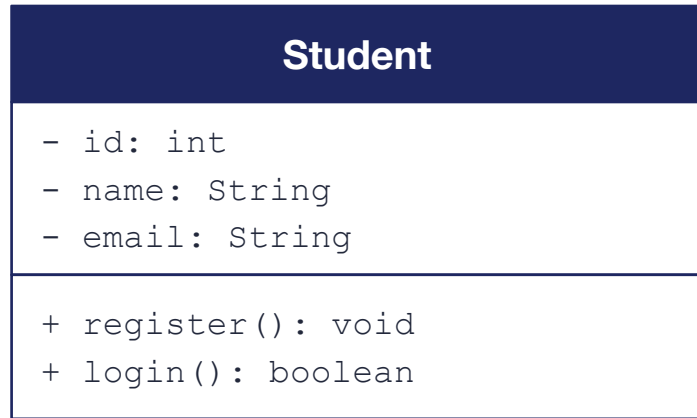
Refactoring

Small structural changes to existing code: same behavior, better design. Usually incremental.

Rewriting

Building a new implementation, potentially with a different architecture altogether.

UML Refresher: Class Diagrams



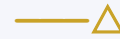
+ *public* - *private* # *protected* ~ *package*

Relationships

Association



Inheritance



Aggregation



Composition



Dependency



Sequence diagrams show who calls whom, and in what order: e.g. Customer → places order → Order → pays → Payment.

MINI CASE STUDY

A Poorly Designed Ordering System

```
class Order {  
    void process(String paymentType) {  
        if (paymentType.equals("CARD")) {  
            // card payment  
        } else if (paymentType.equals(  
            "BKASH")) {  
            // bKash payment  
        }  
        // save order, generate invoice,  
        // send email, send SMS  
    }  
}
```

Discuss in pairs:

- How many responsibilities does Order have?
- Which SOLID principles are violated?
- What problems appear as the system grows?
- Which classes could be extracted?
- Could a pattern eventually help: should we jump there now?

Suggested extraction:

Order • PaymentService • InvoiceGenerator • OrderRepository •
NotificationService

Problem-First, Not Pattern-First

✗ Pattern First

- “I learned Factory.”
- “Where can I use Factory?”

Leads to over-engineering and misapplied patterns.

✓ Problem First

- “I have an object-creation problem.”
- “What's causing it?”
- “Which principles apply?”
- “Would Factory Method help?”

This approach produces better software design.

Benefits vs. Limitations of Patterns

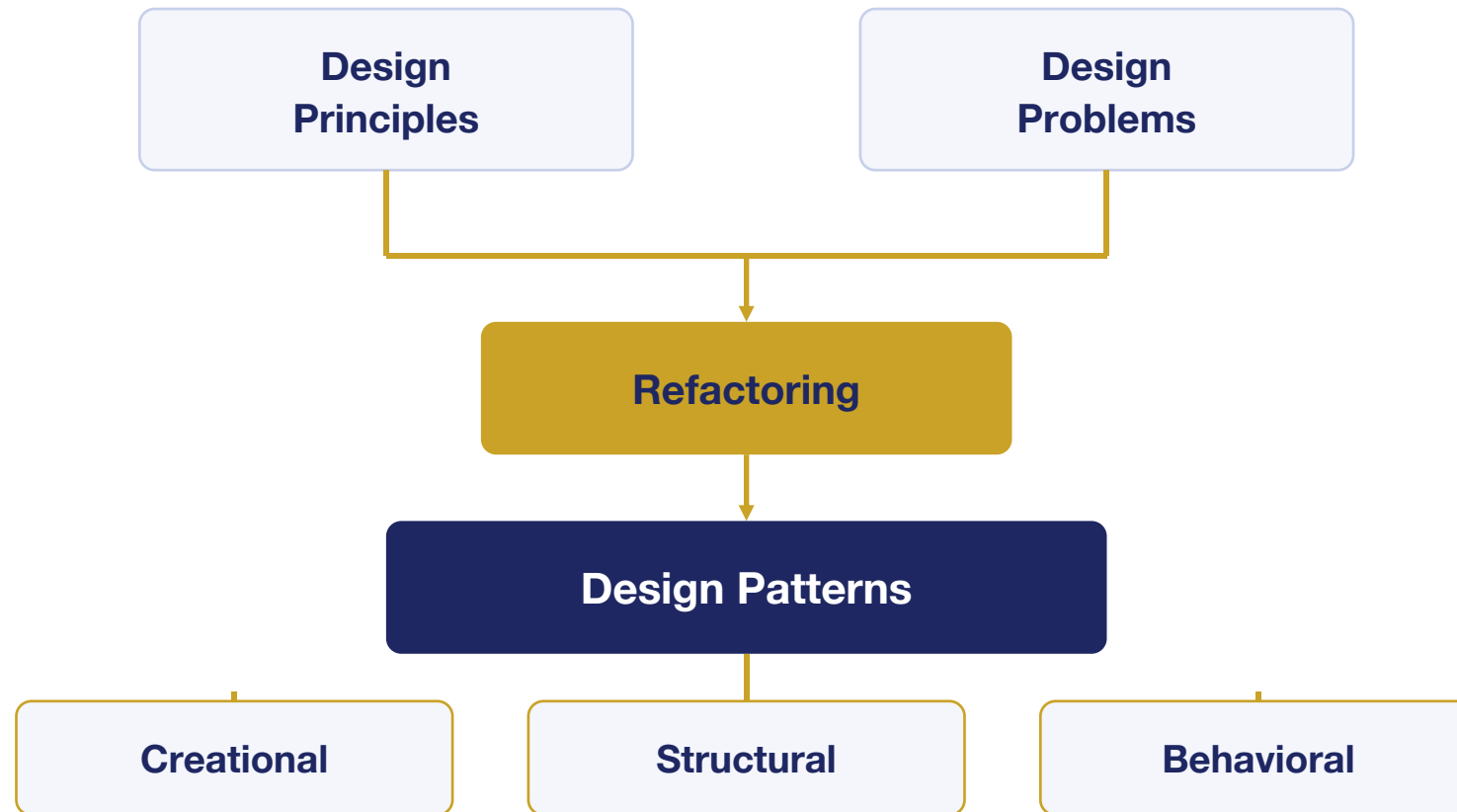
Benefits

- Reusability of proven design ideas
- A common vocabulary between developers
- Improved maintainability
- Greater flexibility to new requirements
- Clearer communication about architecture

Limitations

- Overengineering: 5 classes where 1 would do
- Added abstraction and complexity
- Misapplication to the wrong problem
- A learning curve for maintainers
- A false sense of quality

Summary



Problem → Principles → Design → Refactoring → Pattern (never the reverse)

Questions?

Object-Oriented Design Review coupling, cohesion, composition vs. inheritance.

Md Saeed Siddik

Assistant Professor, IIT, University of Dhaka • Software Design Pattern